PlayPal

A Social Platform for Game Tracking and Discovery

Final Year Project Report

Student Name: Kamil Szlufik

Student ID: 15377821

Email: 15377821@student.setu.ie

Supervisor: Mujanid

Programme: HDip in Science in Computer Science

Academic Year: 2025/2026

South East Technological University

Declaration of Authenticity

I hereby declare that this project report, entitled PlayPal: A Social Platform for Game Tracking and Discovery, is entirely my own work and has not been submitted for assessment in any other module or programme of study. All sources used in the preparation of this report have been properly cited and referenced in accordance with Harvard referencing style.

I acknowledge that the examiners may use plagiarism detection software to check the originality of the submitted work. I confirm that the work submitted is my own and that any assistance received from others has been appropriately acknowledged.

Signed: Kamil Szlufik

Date: March 2026

AI Usage Declaration

During the development and documentation of this project, Claude (Anthropic, 2025), a large language model AI assistant, was used as a development aid throughout various stages of the project. The specific ways in which it was used are described below.

During development, the AI was used to assist with Flutter widget implementation, Firebase integration, and Cloud Functions code for the Steam API proxy. When encountering bugs or unexpected behaviour, the AI was used as a debugging aid in a similar way to how a developer might search Stack Overflow or consult official documentation. For example, when the Firestore permission denied error was occurring on signup due to auth token propagation delays, the AI helped identify the root cause and suggested the delaybased fix. Similarly when the activity feed was silently failing due to a missing composite index the aI helped interpret the Firestore error and explain what needed to be done in the Firebase Console.

The AI was also used as a learning resource when encountering unfamiliar concepts. Dart and Flutter were both new technologies for this project, and the AI was used to explain concepts like StatefulWidget lifecycle, async/await patterns in Dart, and Firestore's documentsubcollection model in practical terms relevant to what was being built. This is comparable to how a student might use a textbook, tutorial, or ask a more experienced colleague for an explanation.

For the report, the AI was used to improve the clarity, structure and academic tone of written sections. All ideas, decisions, requirements, design choices and reflections contained in this report are my own. The AI did not generate the ideas or make the decisions those came from genuine engagement with the problem and the technology throughout the project.

In summary, the AI acted as a tool to accelerate problem solving and learning, not as a replacement for understanding. Every technical decisin discussed in this report was understood, justified, and owned by myself

Acknowledgements

I would like to thank my two friends, both working as software engineers at Smartbox, who gave up their own time to help me understand frontend development concepts and get to grips with the Dart programming language. Their practical industry knowledge was genuinely invaluable and without their input the quality of the final application would have been significantly lower. They answered questions at all hours and never made me feel like I was asking something stupid, which meant a lot during the more frustrating parts of development.

I would also like to thank the open source Flutter and Firebase communities whose documentation, GitHub issues, and Stack Overflow answers helped me resolve technical problems throughout the project.

Abstract

PlayPal is a web application built using Flutter and Firebase that brings together the main things a gamer would want in one place tracking games, discovering new ones, connecting with friends, and discussing games with other users. The system uses the RAWG API to pull in game data, Firebase Authentication for login and account management, Cloud Firestore for storing user data in real time, and Firebase Cloud Functions to securely connect to the Steam Web API so users can import their Steam libraries. There is also a social feed showing what friends have been playing or reviewing, pergame discussion forums where users can post and like comments, and a full friends system with requests and profile pages. The app is deployed at https://playpalapp123.web.app and was built using a single Flutter codebase meaning it can be extended to mobile platforms in the future.

Keywords: Flutter, Firebase, Firestore, Cloud Functions, RAWG API, Steam API, Dart, Node.js, Web Application, Social Platform, Game Tracking

Table of Contents

1. Declaration of Authenticity
2. AI Usage Declaration
3. Acknowledgements
4. Abstract
5. Table of Contents
6. List of Tables
7. List of Figures
8. Glossary
9. List of Abbreviations
10. Preface
11. Background
12. Introduction
13. Methodology (RAMP)
14. Research
15. Analysis
16. Model
17. Plan
18. Implementation
19. Evaluation and Reflection
20. Conclusion and Future Work
21. References
22. Appendices

List of Tables

Table 1: Competitor Platform Analysis

Table 2: Frontend Framework Comparison

Table 3: Backend Technology Comparison

Table 4: Database Technology Comparison

Table 5: External API Comparison

Table 6: MoSCoW Feature Prioritisation

Table 7: Risk Analysis

List of Figures

Figure 1: System Architecture Diagram

Figure 2: Firestore Collection Structure

Figure 3: Development Phase Timeline

Glossary

Term	Definition
Flutter	A crossplatform UI framework developed by Google that allows building applications for web, iOS, Android and desktop from a single Dart codebase (Flutter, 2018)
Firebase	A cloud platform developed by Google providing authentication, database, hosting and serverless function services (Google Firebase, 2014)
Firestore	A NoSQL document database provided by Firebase with realtime data synchronisation capabilities, released in 2017 (Google Cloud, 2017)
Cloud Functions	Serverless Node.js functions hosted on Firebase that run in response to events or HTTP requests (Google Firebase, 2014)
RAWG API	A REST API providing access to a database of over 500,000 video games with metadata including artwork, genres and platforms (RAWG, 2019)
Steam Web API	An official REST API provided by Valve Corporation for accessing Steam user data including game libraries and achievements, available since 2011 (Valve Corporation, 2011)
Widget	The basic building block of a Flutter user interface, equivalent to a component in other frameworks (Flutter, 2018)
Firestore Stream	A realtime listener on a Firestore collection or document that triggers a UI rebuild whenever data changes (Google Cloud, 2017)
Composite Index	A Firestore index that combines multiple fields to support complex query operations (Google Cloud, 2017)
CORS	CrossOrigin Resource Sharing, a browser security mechanism that restricts web pages from making requests to a different domain unless explicitly permitted (MDN Web Docs, 2024)
StatefulWidget	A Flutter widget that can rebuild itself in response to state changes triggered by calling setState() (Flutter, 2018)
MoSCoW	A prioritisation technique dividing requirements into Must Have, Should Have, Could Have and Won't Have categories (Clegg and Barker, 1994)

List of Abbreviations

Abbreviation	Full Term
API	Application Programming Interface
CORS	CrossOrigin Resource Sharing
UI	User Interface
UX	User Experience
RAWG	rawg.io Video Games Database
NoSQL	Not Only SQL
SQL	Structured Query Language
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
SDK	Software Development Kit
IDE	Integrated Development Environment
RAMP	Research, Analysis, Model, Plan
MoSCoW	Must Have, Should Have, Could Have, Won't Have
HDip	Higher Diploma
SETU	South East Technological University
AI	Artificial Intelligence
LLM	Large Language Model

Preface

This report documents the design, development and evaluation of PlayPal, a social game tracking and discovery platform built as the final year project for the Higher Diploma in Science in Computer Science at South East Technological University.

The motivation for this project came from a personal frustration with how fragmented the gaming experience is across different platforms and tools. As someone who plays games across multiple platforms and likes to keep track of what has been played and what friends are playing, constantly switching between different apps with none of them doing everything needed was a real problem. PlayPal was an attempt to build something that actually solved that problem, at least as a prototype.

The project also provided an opportunity to learn Flutter and Firebase, two technologies not covered in the course. Flutter, described by Google as a UI toolkit for building natively compiled applications from a single codebase (Flutter, 2018), turned out to be an excellent choice for this kind of project. Both technologies were challenging to learn but rewarding to work with.

This report is written in the past tense and describes what was built, the decisions made along the way, and an honest reflection on what went well and what could have been done better. The live application is available at https://playpalapp123.web.app and the source code is at https://github.com/Kszlufik/FinalProject.

1. Background

The global video games market has experienced significant growth over the past decade. According to Newzoo (2023), the global games market generated revenues of over $184 billion in 2023, with players engaging across PC, console, and mobile platforms. This growth has led to increasingly large gaming libraries and a corresponding need for tools to manage, track, and discuss gaming activity.

The problem is that none of these platforms really talk to each other. If you want to keep track of what you have played, what you want to play, and what friends are doing, you end up juggling a bunch of different apps and websites. Some people use spreadsheets, some use dedicated tracking sites like Backloggd (Backloggd, 2024), others just post on Reddit or Discord. None of it is joined up.

There are tools out there that try to help with this but most of them only solve one piece of the puzzle. Sites like Backloggd are great for tracking but have almost no discovery features. RAWG.io is excellent for finding games but does not have any personal tracking (RAWG, 2019). Steam, developed by Valve Corporation, is the closest thing to a complete solution for PC gamers but it only works for Steam games and is locked to that ecosystem (Valve Corporation, 2011). This fragmentation is what motivated this project.

From a technical perspective this project was also an opportunity to explore Flutter, which was not covered in the course. Flutter is described as an open source UI software development kit created by Google for building natively compiled, multiplatform applications from a single codebase (Flutter, 2018). The combination of Flutter with Firebase's real time cloud services (Google Firebase, 2014) turned out to be very well suited to this kind of social application.

2. Introduction

PlayPal is a social gaming platform built as a final year project. The goal was to design and build something that a gamer could actually use day to day, rather than just a demo with fake data. The app allows users to search for games using the RAWG API (RAWG, 2019), save favourites, write reviews and track play status, connect their Steam account to import their library (Valve Corporation, 2011), add friends and see what they have been playing, join discussions on individual game pages, and view a social activity feed showing friends' recent activity.

The application runs as a web app built with Flutter Web (Flutter, 2018) and hosted on Firebase Hosting (Google Firebase, 2014). Steam library data is fetched through Firebase Cloud Functions which act as a proxy to the Steam Web api this was necessary because the Steam API does not support CORS, a browser security mechanism that restricts crossorigin HTTP requests (MDN Web Docs, 2024), which would have blocked requests coming directly from the browser.

The project is intentionally scoped as a prototype. Some features that would exist in a production application are not included things like push notifications, private messaging, and recommendation algorithms, and perhaps the most important is a true, crossplatoform integration where we have gamer data from Playstation, Xbox ,Nintendo and the like. But the core functionality works end to end and is deployed publicly, which was an important goal from the start.

3. Methodology (RAMP)

This project used the RAMP methodology which stands for Research, Analysis, Model and Plan. RAMP is a structured approach to software development that emphasises understanding the problem domain before committing to design and implementation decisions. The methodology is well suited to academic projects where documented justification of decisions is as important as the technical outcome itself.

The Research phase involved investigating existing competitor platforms and the technologies available to build something similar. The Analysis phase turned those findings into actual requirements using MoSCoW prioritisation, a technique developed by Clegg and Barker (1994) that categorises requirements into Must Have, Should Have, Could Have and Won't Have. The Model phase was where the architecture, data structures and UI concepts were designed before any code was written. The Plan phase broke everything down into manageable sprints and identified the risks.

In practice the phases were not perfectly sequential. There was quite a bit of going back to update the model or adjust requirements as the implementation progressed and new technical constraints became clear. But having the framework in place made it easier to make those adjustments in a structured way rather than just making things up as you go.

4. Research

4.1 Domain Research

The starting point was understanding what already exists in this space. According to Statista (2023), there were approximately 3.38 billion gamers worldwide in 2023, a figure that highlights the scale of the potential user base for a game tracking and social platform. Gaming communities are very active online and there is a lot of discussion about tools like Backloggd (Backloggd, 2024), HowLongToBeat (HowLongToBeat, 2024), and Steam (Valve Corporation, 2011). The main themes that came up repeatedly in community discussions were the desire for cross platform tracking, a clean interface, and a social element so you can see what friends are playing.

4.2 Competitor Analysis

A number of existing platforms were reviewed to understand the current landscape and identify where PlayPal could offer something different. The platforms selected were chosen based on relevance and feature overlap with the proposed system.

Table 1: Competitor Platform Analysis

Platform	Core Features	Strengths	Limitations
PlayTracker (PlayTracker, 2024)	Game library, playtime logging, backlog	Simple interface, focused tracking	Limited social features, no discussion forums
Backloggd (Backloggd, 2024)	Backlog tracking, ratings, reviews	Strong community, clean UI	Limited metadata automation, web only
RAWG (RAWG, 2019)	Game discovery, metadata, ratings	Large database, strong API	Not a personal tracker, no social tools
Steam (Valve Corporation, 2011)	Library, achievements, social	Deep PC ecosystem integration	Locked to Steam platform only
Xbox/PlayStation	Play history, achievements, friends	Accurate platform data	Closed ecosystems, very limited customisation

What became clear from this analysis is that no single platform covers everything. PlayPal's opportunity was to combine the tracking and discovery features of these tools with a genuine social layer and community discussion, all in one place.

4.3 Technology Research

Choosing the right technologies was one of the more important early decisions. The main requirements were cross platform capability, a backend that could handle real time data, and a stack manageable by a single developer without too much infrastructure overhead.

4.3.1 Frontend Frameworks

Table 2: Frontend Framework Comparison

Framework	Description	Advantages	Limitations
Flutter (Selected)	Crossplatform UI toolkit by Google (Flutter, 2018)	Single codebase for web, iOS, Android, desktop. Strong UI control	Dart learning curve, smaller ecosystem than React
React Native (Meta, 2015)	JavaScript mobile framework	Familiar to web developers, large community	Web support limited, requires Expo or separate tooling
Kotlin Multiplatform (JetBrains, 2020)	Shared business logic across platforms	Good native performance	UI still needs separate implementation per platform

Flutter was selected because of its ability to deliver a single consistent user interface across all target platforms from one codebase (Flutter, 2018). This aligned directly with the goal of reducing fragmentation and demonstrating modern multiplatform development. The Dart language took time to learn but friends who work as software engineers at Smartbox were particularly helpful in explaining Flutter's widget tree and state management patterns.

4.3.2 Backend and Authentication

Table 3: Backend Technology Comparison

Technology	Description	Advantages	Limitations
Firebase (Selected)	Google cloud platform (Google Firebase, 2014)	Integrated auth, real time database, serverless functions, generous free tier	Vendor lockin, less control over backend logic
Supabase (Supabase, 2020)	Open source Firebase alternative with PostgreSQL	Relational database, open source	Less mature Flutter SDK
Node.js + Express (Node.js Foundation, 2009)	JavaScript runtime and web framework	Flexible, well documented	Requires separate hosting, more infrastructure overhead

Firebase was chosen primarily because of its seamless integration with Flutter and its generous free tier (Google Firebase, 2014). Firestore's real time synchronisation capabilities (Google Cloud, 2017) were particularly valuable for features like the discussion forums and activity feed. Firestore is great for a project of this type, especially if the priority is delivering a complete product without getting too bogged down in the intricacies of a custom backend. Looking back however, using PostgreSQL (PostgreSQL Global Development Group, 1996) with a custom Express backend would have been more impressive technically and would have provided more control over the

backend logic. If thinking of this as a real production product, PostgreSQL would be the stronger choice.

4.3.3 Database Technologies

Table 4: Database Technology Comparison

Database	Type	Advantages	Limitations
Cloud Firestore (Selected)	NoSQL document (Google Cloud, 2017)	Real time sync, flexible schema, native Firebase integration	NoSQL limits complex relational queries
PostgreSQL (PostgreSQL, 1996)	Relational SQL	Excellent structured data support, normalisation, full SQL	Needs separate hosting, no built in real time
MongoDB (MongoDB, 2009)	NoSQL document	Flexible schema, JSONlike storage	No built in real time, separate hosting needed

Firestore was chosen partly because it comes with Firebase but also because its document model suited the hierarchical ownership structure of this application well (Google Cloud, 2017). Each user owns subcollections of favourites, reviews, and friends, which maps naturally to Firestore's documentsubcollection pattern. The main constraint encountered was that complex queries combining filtering and ordering require manually created composite indexes in the Firebase Console, something that was not immediately obvious during development.

4.3.4 External APIs

Table 5: External API Comparison

API	Purpose	Strengths	Limitations
RAWG.io (Selected)	Game metadata and discovery (RAWG, 2019)	500,000+ games, free tier, REST API with CORS support	Rate limits on the free tier
Steam Web API (Selected)	Steam library and achievements (Valve Corporation, 2011)	Official data, accurate playtime and achievements	No CORS support, requires server side proxy for web apps
IGDB (Twitch, 2015)	Game metadata	Very detailed data, Twitch integration	Requires OAuth authentication, more complex setup
Giant Bomb (Giant Bomb, 2012)	Game database	Good community content	Limited free access, less actively maintained

RAWG was the clear choice for game discovery given its large catalogue and REST API with native CORS support, meaning it could be called directly from the Flutter web client without issues (RAWG, 2019). Steam was included to add the library import feature. The Steam Web API however does not include the CORS headers required by browsers for crossorigin requests (MDN Web Docs, 2024), meaning direct calls from a web application are blocked by the browser's security policy before they ever reach the server. The solution was to proxy all Steam API calls through Firebase Cloud Functions

(Google Firebase, 2014), which run server side in Node.js and are not subject to browser CORS restrictions. The Flutter client calls the Cloud Function over HTTPS, the function calls the Steam API from the server, and returns the result to the client. This a

5. Analysis

5.1 Functional Requirements

- The system shall allow users to register and log in using email and password via Firebase Authentication
- The system shall allow users to search for games using the RAWG API and retrieve metadata including title, artwork, genres, release date and platforms
- The system shall allow users to save games to a personal favourites list.
- The system shall allow users to set a play status for games (Playing, Completed, Dropped).
- The system shall allow users to write personal reviews with a star rating from 1 to 5 and optional text notes.
- The system shall allow users to connect their Steam account and view their game library and achievements
- The system shall allow users to search for other users and send, accept or decline friend requests.
- The system shall allow users to view a friend's profile showing their reviews, favourites and gaming stats.
- The system shall provide a social activity feed showing recent reviews and favourites from friends.
- The system shall provide per game discussion forums where users can post comments, like posts and delete their own comments.
- The system shall provide a personal profile page with an editable username, bio and avatar colour.
- The system shall allow users to reset their password via email.

5.2 NonFunctional Requirements

- The system shall be deployed as a publicly accessible web application.
- The interface shall be responsive and work on both desktop and mobile screen sizes.
- A consistent visual design shall be maintained across all screens.
- All user data shall be protected by Firestore security rules requiring authentication
- Steam API credentials shall never be exposed to the client, all Steam requests shall go through Firebase Cloud Functions
- Forum posts and activity feed entries shall update in real time using Firestore stream listeners

5.3 MoSCoW Feature Prioritisation

Features were prioritised using the MoSCoW technique (Clegg and Barker, 1994) which divides requirements into four categories based on their importance and feasibility within the available time.

Table 6: MoSCoW Feature Prioritisation

Priority	Features
Must Have	Authentication, game search via RAWG API, favourites library, play status tracking, reviews and ratings, Steam library integration, friends system, personal profile, mobile responsive layout, social activity feed, pergame discussion forums
Should Have	Password reset by email, sign out confirmation dialog, recently viewed games strip, avatar colour customisation
Could Have	Private messaging between users, game recommendations based on history, real time notifications, iOS and Android app deployment
Won't Have	Advanced analytics, monetisation features, third party OAuth login

5.4 Happy Path

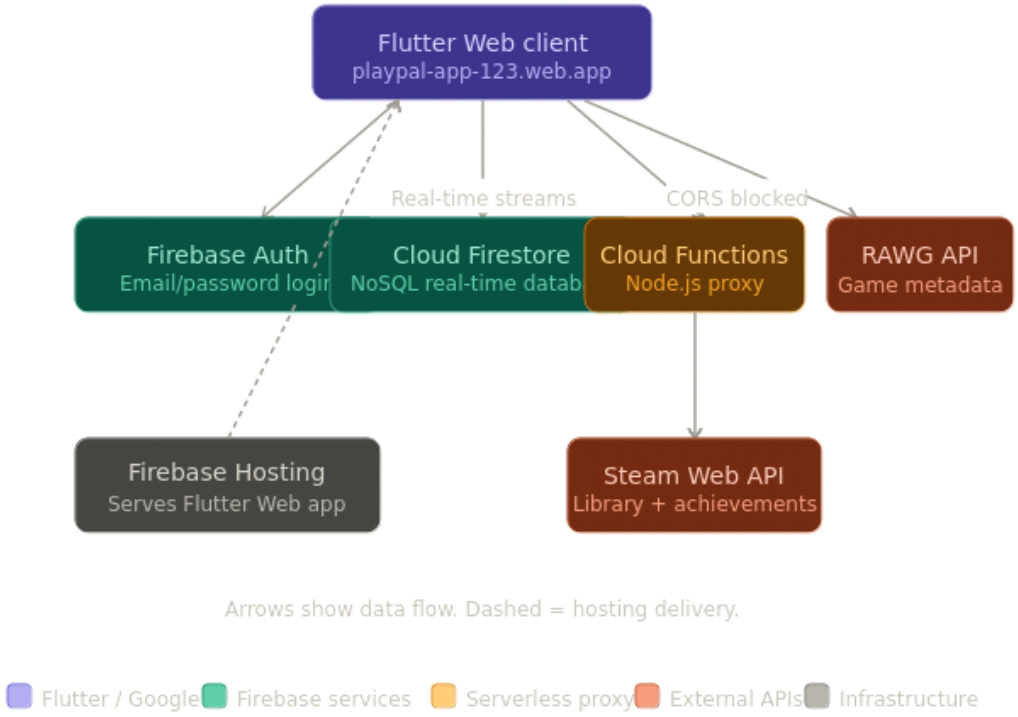
The happy path for PlayPal is: a user signs up with a username and email, searches for a game, saves it to favourites, writes a review with a status and star rating, connects their Steam account, adds a friend by username, checks the feed to see what that friend has been playing, and leaves a comment in the discussion forum. That sequence touches every major part of the system and demonstrates that everything works together end to end.

Figure 4: Happy Path Flow Diagram

6. Model

6.1 System Architecture

The architecture of PlayPal is built around Firebase as the central cloud platform (Google Firebase, 2014). The Flutter Web frontend (Flutter, 2018) communicates directly with Firebase Authentication for login and with Cloud Firestore (Google Cloud, 2017) for reading and writing data. Steam API calls go through Firebase Cloud Functions rather than directly from the browser because the Steam API does not support CORS (MDN Web Docs, 2024) and would reject requests from a web page. For the activity feed and discussion forums, Firestore's real time stream functionality (Google Cloud, 2017) is used so that new data appears on screen automatically without any polling or manual refresh.



[

Figure 1: System Architecture Diagram

6.2 Firestore Data Structure

The data is organised in Firestore (Google Cloud, 2017) as a set of collections and subcollections. User specific data lives under each user's document in subcollections, and shared data like the activity feed and forums lives at the top level.

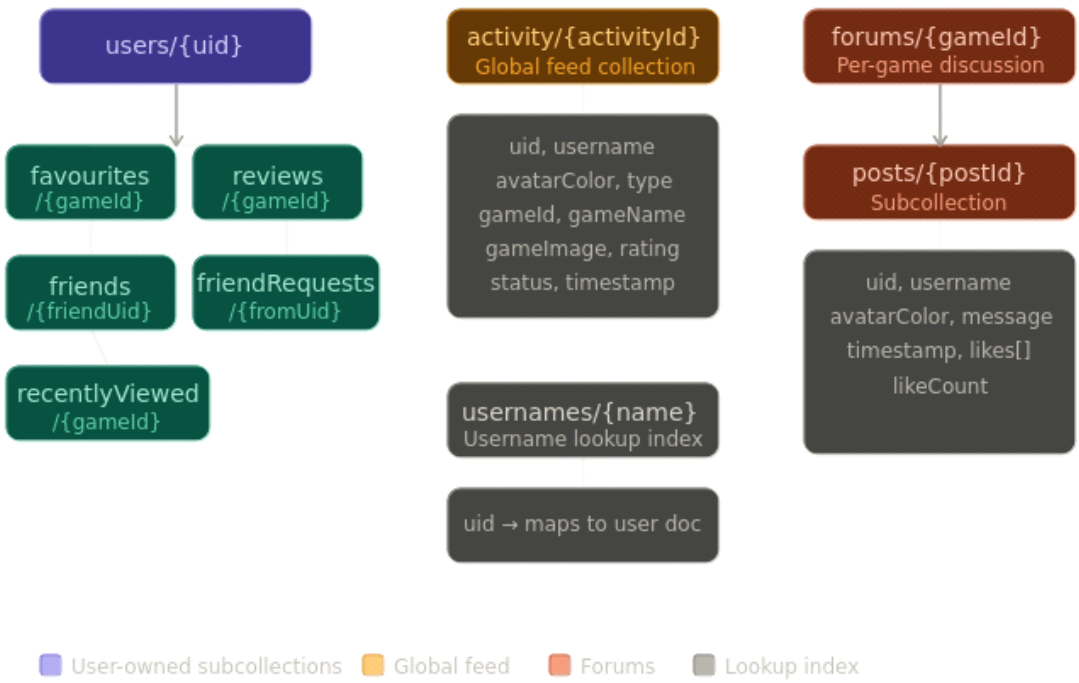


Figure 2: Firestore Collection Structure

6.3 UI Design System

The visual design uses a dark colour scheme throughout, inspired by platforms like Steam and GitHub's dark mode. The main colours are a near black background (#0D1117), dark surface cards (#161B22), a cyan accent (#00E5FF) for interactive elements, and muted grey for secondary text. Star ratings use gold and status indicators use green for completed, cyan for playing, and red for dropped.

The layout switches at 900px following responsive design principles (MDN Web Docs, 2024). Above that width a 220px left panel is always visible. Below 900px the panel is replaced by a slide out drawer accessible via a hamburger menu button, following established mobile navigation patterns (Google Material Design, 2021).

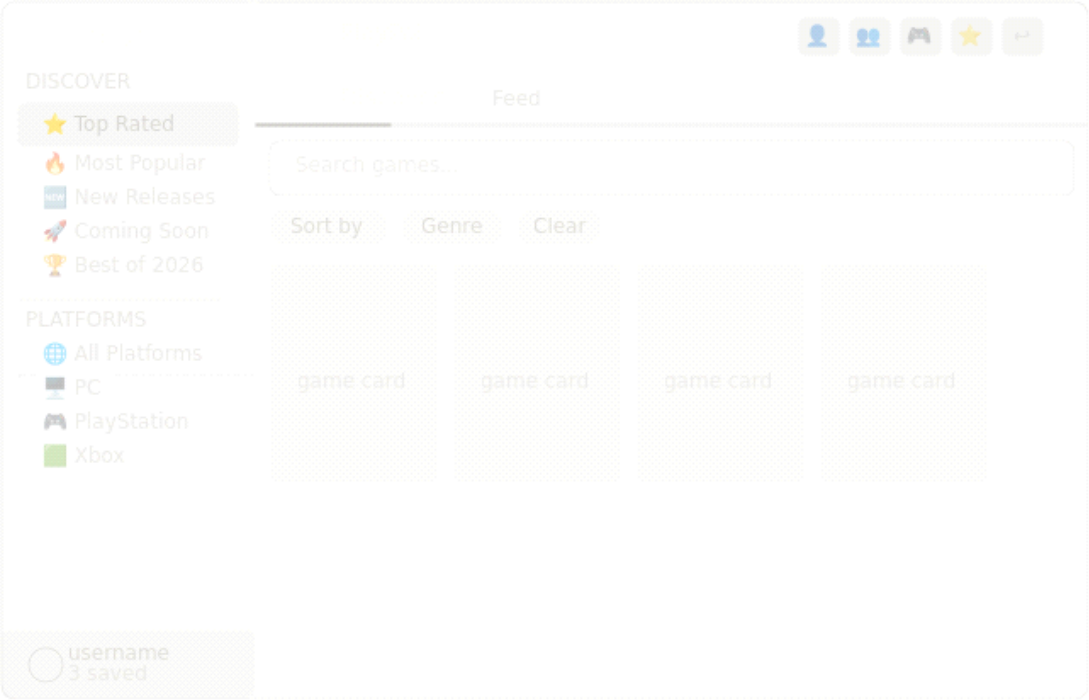


Figure 5: Home Screen Wireframe

Figure 6: Login Screen Wireframe

7.1 Risk Analysis

Table 7: Risk Analysis

Risk	Likelihood	Impact	Mitigation
CORS errors with Steam API	High	High	Route all Steam calls through Firebase Cloud Functions (Google Firebase, 2014)
Firebase token propagation delay on signup	Medium	High	Add 2 second delay after account creation before first Firestore write
Firestore composite index requirements	Medium	Medium	Create indexes in Firebase Console when query errors occur (Google Cloud, 2017)
Overscoping features	High	Medium	MoSCoW prioritisation (Clegg and Barker, 1994), finish one feature before starting next
Steam API key exposure	Low	High	Store as Firebase Secret, never in source code (Google Firebase, 2014)

7.2 Sprint Structure

Development was organised into informal two week sprints following agile principles (Beck et al., 2001). Each sprint had a clear goal and a small set of features to complete before moving on.

- Sprint 1: Firebase setup, authentication, basic Flutter project structure
- Sprint 2: RAWG API integration, game search, favourites
- Sprint 3: Reviews system, game details screen, UI design system
- Sprint 4: Steam integration via Cloud Functions
- Sprint 5: Friends system, friend profiles
- Sprint 6: Activity feed and discussion forums
- Sprint 7: Mobile responsiveness, deployment, final polish

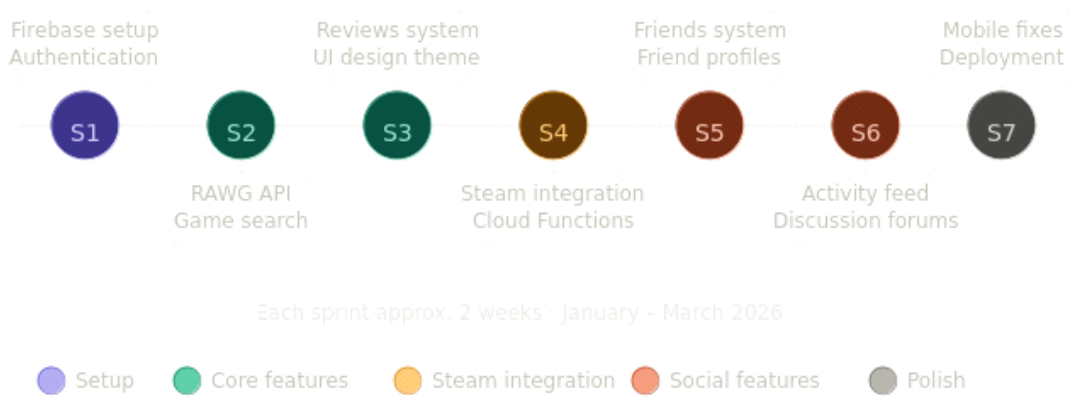


Figure 3: Development Phase Timeline

8. Implementation

8.1 Development Environment

- Frontend: Flutter 3.x with Dart (Flutter, 2018), targeting Flutter Web
- Backend: Firebase Cloud Functions, Node.js 24 (Node.js Foundation, 2009), 2nd generation
- Database: Cloud Firestore (Google Cloud, 2017)
- Authentication: Firebase Authentication (Google Firebase, 2014)
- Hosting: Firebase Hosting at https://playpalapp123.web.app
- External APIs: RAWG.io (RAWG, 2019) and Steam Web API (Valve Corporation, 2011)
- Version Control: Git and GitHub at https://github.com/Kszlufik/FinalProject
- AI Development Aid: Claude (Anthropic, 2025)

8.2 Authentication

Login and registration were handled by Firebase Authentication (Google Firebase, 2014). The signup process collected a username, email and password. The username was checked for uniqueness by querying the Firestore usernames collection (Google Cloud, 2017) before the account was created. One issue encountered was a permission denied error immediately after creating a new auth account. The cause was that Firebase's authentication token takes a short time to propagate after account creation, and the Firestore write was happening before the token was ready. The fix was to add a short delay after calling createUserWithEmailAndPassword before attempting the first Firestore write.

8.3 Game Discovery

Game search and browsing was powered by the RAWG API (RAWG, 2019). The RawgService class handled all API communication, supporting filtering by genre, platform, date range and search query. The home screen used infinite scroll, loading the next page of results automatically as the user reached the bottom of the game grid. Games were shown in a responsive grid that adjusted the number of columns based on screen width following responsive web design principles (MDN Web Docs, 2024).

8.4 UI Design and Theme

One of the most challenging and time consuming parts of the project was getting the visual design right. For a long time the app felt generic and empty even with all the features working. A lot of time was spent experimenting with different colour combinations, spacing, and component styles before settling on the final design system. The cyan accent on a nearblack background, combined with a cardbased layout and consistent border styling, was what finally made the app feel like a real product. Getting that right took significantly longer than expected and involved a lot of iteration.

8.5 Steam Integration

The Steam integration was the most technically complex feature. It required writing three Cloud Functions (Google Firebase, 2014) that each handled a different Steam API endpoint (Valve Corporation, 2011), deploying them to Firebase, and storing the API key as a Firebase Secret. The trickiest parts were: parsing Steam profile URLs since users might paste their full profile URL, their vanity URL, or just their numeric Steam ID; and the getSteamAchievements function which had to call two different Steam endpoints in parallel using Promise.all in Node.js (Node.js Foundation, 2009) and merge the results, because one endpoint returned the user's unlock data and the other returned display names and icons for the achievements.

The CORS issue (MDN Web Docs, 2024) was the first major technical obstacle with the Steam integration. Because PlayPal runs in a browser, every HTTP request is subject to CORS restrictions. The Steam Web API does not include the AccessControlAllowOrigin header that browsers require before allowing a crossorigin response. This meant every direct call to the Steam API from Flutter was silently blocked by the browser. Routing all Steam calls through Firebase Cloud Functions (Google Firebase, 2014) resolved this completely since the functions run server side and are not subject to browser security policies.

8.6 Activity Feed

The feed showed recent reviews and favourites from the current user's friends. When either of those actions happened, a document was written to the global activity Firestore collection (Google Cloud, 2017) with the user's details, the game info, and a server timestamp. The feed screen loaded the user's friend list first, then queried the activity collection using a whereIn filter on the list of friend UIDs. This query required a composite Firestore index (Google Cloud, 2017) which had to be created manually in the Firebase Console. Once the index was built the feed worked correctly.

8.7 Discussion Forums

Each game had its own discussion thread stored at forums/{gameId}/posts in Firestore (Google Cloud, 2017). The forum screen used a StreamBuilder with a real time Firestore stream listener, which meant new posts appeared on screen immediately without any polling or manual refreshing. Liking a post used Firestore's arrayUnion and arrayRemove operations to add or remove the user's UID from the likes field. The post with the highest likeCount was identified client side and received a gold trophy badge labelled Top Comment.

8.8 Mobile Responsiveness

The app originally had a black screen issue on iPhone Safari when accessed as the deployed web app. On web we have wider screens meaning the top navigation buttons are spread comfortably across the bar, but on phones the same buttons were squeezed together in a much narrower space. This had not been fully anticipated during initial development, causing Flutter widgets to overflow on top of each other which produced the black screen. After accounting for narrow screen widths and implementing layout adjustments reducing button padding, scaling icon sizes based on screen width, and forcing the HTML renderer in the Flutter web configuration (Flutter, 2018) the issue was fully resolved. The app was tested and verified working on iPhone SE which at 375px wide is one of the narrowest screen sizes in common use.

9. Evaluation and Reflection

9.1 Requirements Review

All Must Have and Should Have features from the requirements analysis were implemented and are working in the deployed application. The Could Have features private messaging, game recommendations, real time notifications, and mobile app deployment were intentionally deferred as they were always considered out of scope for a prototype.

9.2 What Went Well

The overall outcome of the project exceeded what was originally planned. The app genuinely looks and feels like a real product rather than a student project, which was something that mattered a lot throughout development. The dark theme with the cyan accent on the nearblack background gives it a visual identity that feels appropriate for a gaming platform.

The social layer the friends system, the activity feed, and the discussion forums all came together better than expected. Having the feed update in real time using Firestore streams (Google Cloud, 2017) was particularly satisfying. The Steam integration also worked out well in the end despite being difficult, and having real Steam library data loading into the app made the whole thing feel much more useful and complete.

The fact that the app works on both desktop and mobile and is publicly deployed and accessible to anyone was another goal that was fully achieved. Being able to demonstrate a live working application rather than just a local demo is something genuinely worth being proud of.

9.3 Problems Encountered

The biggest challenge was the visual design. For a long time the app had all the features but looked boring and generic. Finding the right combination of colours, spacing and component styling took much longer than any individual feature and involved a lot of trial and error. It is easy to underestimate how much work goes into making something look genuinely good.

The Steam integration was the most technically difficult feature. The CORS problem (MDN Web Docs, 2024) meant all Steam calls had to go through Cloud Functions (Google Firebase, 2014). Parsing multiple Steam URL formats required careful string handling, and merging data from two different Steam endpoints in parallel required a solid understanding of Promise.all in Node.js (Node.js Foundation, 2009).

The Firestore composite index requirements (Google Cloud, 2017) were also a source of frustration. Firestore handles simple queries easily but combining filtering on one field with ordering on another requires a composite index that must be created manually in the Firebase Console. The error messages do include a direct link to create the index

but it caused unexpected delays both for the activity feed and the discussion forum queries.

The black screen on iPhone was another unexpected problem. On web wider screens give the navigation buttons plenty of room, but on phones the same layout tried to squeeze those buttons into a 375px wide bar, causing widgets to overflow on top of each other and producing a blank screen. After accounting for this and implementing responsive adjustments to padding, icon sizes and the Flutter web renderer configuration, the issue was fully resolved.

9.4 What I Would Do Differently

The main thing that would be changed if starting again is the backend. The primary reason for choosing Firebase (Google Firebase, 2014) was time building a solid front end whilst simultaneously designing and managing a custom backend would have delayed the project considerably to the point where not all functional requirements might have been met. Firebase made it possible to move quickly without those concerns. That said, using PostgreSQL (PostgreSQL Global Development Group, 1996) and building a proper Node.js Express backend (Node.js Foundation, 2009) would have been more impressive technically and would have taught significantly more about database design, SQL, migrations, and REST API development.

A proper state management solution like Riverpod would also have made some of the more complex screens cleaner. Using setState throughout the app worked fine for most things but in screens that needed to share state across multiple widgets it got messy. Planning the data model more carefully at the start of the project would also have saved some refactoring time later.

9.5 Technical Evaluation

Flutter (Flutter, 2018) was a good choice and would be used again for a cross platform project. The widget system is powerful and once the patterns click it is fast to work with. Friends who work as software engineers at Smartbox were particularly helpful in explaining Dart's async programming model and Flutter's StatefulWidget lifecycle, which were quite different from anything covered in the course.

Firebase (Google Firebase, 2014) did everything it needed to do for this project. The real time capabilities of Firestore (Google Cloud, 2017) were particularly well suited to the discussion forums and feed features. The main limitation is the NoSQL data model which makes some kinds of queries quite difficult, and vendor lockin means migrating away from Firebase in the future would be a significant undertaking.

10. Conclusion and Future Work

10.1 Conclusion

PlayPal started as an idea to solve a problem that was personally experienced the fact that there is no single place to track games, see what friends are playing, and talk about games with other people. The idea felt simple enough at the start, but as the project grew it became something much bigger than originally planned.

The early weeks were more difficult than anticipated. Flutter and Firebase (Flutter, 2018; Google Firebase, 2014) were both completely new technologies, and there were moments in those first few weeks where the learning curve felt genuinely steep. Getting authentication working correctly, understanding Firestore's documentsubcollection model, and wrapping your head around Flutter's widget tree and reactive state management all at the same time is a lot to take on simultaneously. There were days where progress felt slow and the gap between what the app looked like and what it was supposed to look like felt discouraging. Looking back, that period was where most of the real learning happened. The frustration of not understanding something and then finally breaking through it is a different kind of learning to reading documentation or following a tutorial, and it builds a different kind of confidence.

There was a specific moment during development that felt like a turning point. When the Steam integration (Valve Corporation, 2013) finally worked when real game library data, playtime statistics, and achievements were loading from an actual Steam account directly into the app that was the moment it became clear that the rest could be done too. Before that, the project felt like an ambitious plan. After that, it felt like a real product being built. The Steam integration was also one of the more technically involved parts of the project, requiring Firebase Cloud Functions to be written in Node.js to proxy requests around the Steam API's lack of CORS support, and getting it to work endtoend felt like a genuine engineering achievement rather than just following a guide.

The activity feed was the most satisfying feature to build because it was what made PlayPal feel like a social platform rather than a personal game tracker. Seeing a friend's review appear in real time in the feed without a page refresh, without any polling, just Firestore's stream listeners doing their job was the moment the app started to feel alive in a way that individual screens never quite do on their own. It is the feature that most clearly demonstrates what the project was trying to be, and getting it right took longer than expected precisely because it touches so many other parts of the system simultaneously.

When the app was first opened on an iPhone after the mobile fixes were in place and the deployment was live, it was a genuinely proud moment. All that time of work had produced something that looked polished, felt smooth, and had every planned feature actually working. It did not look like a student project it looked like something you might actually download. That feeling, of seeing code you wrote running on a real device and looking the way you imagined it, is difficult to describe but it is what made the whole

project worth the effort. Getting the visual design right was unexpectedly one of the hardest problems harder in some ways than any of the individual technical challenges. Picking a colour palette is easy. Making a darkthemed interface feel cohesive, modern, and intentional rather than just dark is a different problem entirely, and it took significantly more iteration than expected before it felt right.

The project was not without its frustrations. The Firestore composite index issue that caused the activity feed to silently fail was one of the more demoralising bugs to track down, because nothing in the app gave any visible indication that something was wrong. The Firebase token propagation delay on signup where the app would attempt to write to Firestore before the authentication token had fully propagated was another issue that took time to diagnose and required an inelegant but pragmatic fix. These are the kinds of problems that do not appear in tutorials and that you only encounter by building something real at a sufficient level of complexity. Solving them without a colleague to ask was one of the more genuinely challenging aspects of the project, and it made the debugging support from friends in industry all the more valuable when it was available.

Beyond being a final year project, PlayPal means something more personally. It proved that with the right tools, the right support, and enough persistence, a complete idea can be taken from nothing to a deployed, working application that real people can use. That confidence knowing the full process, from architecture decisions and database design through to responsive UI, cloud deployment, and thirdparty API integration is the most valuable thing that came out of this project. It is not just technical knowledge; it is the knowledge that the process works and that finishing something difficult is possible. That will carry forward into every project that comes after this one.

All of the Must Have requirements from the analysis phase were implemented and are working in the live application. The goals set at the start of the project were not just met but exceeded in both scope and quality. PlayPal is not a prototype or a proof of concept it is a functioning, deployed web application with real features, real data, and real users able to sign up and use it today. That is what this project set out to build, and that is what it became.

10.2 Future Work

The most natural next steps would be getting the app onto iOS and Android properly. Flutter (Flutter, 2018) makes this relatively straightforward since the same codebase compiles for mobile with some adjustments, and a lot of the responsive layout work done during development would carry over directly. However, mobile deployment is not simply a matter of recompiling. Push notifications would need to be wired up through Firebase Cloud Messaging (Google Firebase, 2014), platformspecific app store submission processes would need to be navigated, and deeper mobile integrations such as haptic feedback, native share sheets, and camera access for profile photos would need to be implemented thoughtfully. Testing across a range of physical devices and screen sizes would also be a significant undertaking that goes well beyond what emulators can cover.

On the social side, the current friends and activity feed system could be meaningfully extended. Reply threads in the discussion forums would make conversations more structured and easier to follow, particularly for popular games with high post volumes. Private messaging between users would bring PlayPal closer to a fully selfcontained social platform and reduce the need to move conversations offapp. A notification system for forum replies, and activity from followed users would improve engagement considerably and is something users would expect from any modern social application. These notifications would need to be implemented carefully to avoid becoming intrusive, with per user preferences for which events trigger alerts and through which channels. The recommendation engine represents the most technically ambitious area of potential future development. The app already collects rich data on user behaviour game ratings, completion statuses, genre preferences, and play history which forms a solid foundation for a collaborative filtering approach (Su and Khoshgoftaar, 2009). A basic implementation could identify users with similar taste profiles and surface games they have rated highly that the current user has not yet played. A more sophisticated version could incorporate contentbased signals from the RAWG API (RAWG, 2021) such as genre, developer, and tags to handle the cold start problem for newer users who have not yet built up enough review history for collaborative filtering to work effectively. This would be a genuinely interesting machine learning problem to work on and would add significant product value.

On the backend, migrating from Firebase to a PostgreSQL database (PostgreSQL Global Development Group, 1996) with a custom Express API would make the system significantly more robust and open up more complex query possibilities that Firestore's document model makes difficult or impossible. Firestore handles the queries PlayPal currently needs well, but as the feature set grows, the limitations of a NoSQL document store become more pronounced
Beyond the larger architectural and feature changes described above, there are a significant number of quality of life improvements that would meaningfully improve the daytoday experience of using PlayPal.

An admin account system with elevated privileges would be an important addition as the platform grows. Currently there is no moderation layer, meaning inappropriate forum posts or spam accounts can only be addressed by going directly into the Firebase Console. A proper admin role would allow trusted users to remove posts, ban accounts, and manage reported content through the app itself without requiring database access. This is standard practice on any communitydriven platform and becomes increasingly necessary as user numbers grow.

The onboarding experience is an area that needs significant rethinking. At the moment a new user who creates an account is immediately presented with the full game library loading in, which can feel overwhelming and impersonal. A much better approach would be a short onboarding flow after signup that asks the user to select a handful of genres they are interested in and pick a few games they have already played or enjoyed. This would serve two purposes simultaneously: it gives PlayPal enough information to start personalising the home screen immediately, and it gives the user a sense of ownership

over their experience before they have had time to explore on their own. Platforms like Letterboxd and Spotify both use a variation of this approach to great effect, and the difference in first impression between a curated starting point and a blank or generic feed is considerable.

The more pressing concern, however, is the current approach to user acquisition and retention at the very top of the funnel. Right now, a visitor to PlayPal is immediately faced with a login or account creation screen. This is one of the least userretentionfriendly patterns in modern web application design. A user who has never heard of PlayPal has no reason to hand over their email address before they have seen anything that demonstrates the platform's value. The common and wellestablished solution to this is to allow guest browsing letting unauthenticated users explore the game library, read reviews, and browse forum discussions before being prompted to create an account. The prompt to sign up should be deferred until the user tries to do something that genuinely requires an account, such as writing a review, adding a game to their library, or sending a friend request. This pattern, sometimes called progressive disclosure of authentication, significantly reduces dropoff at the entry point and is used by virtually every successful consumer platform. Implementing it in PlayPal would require making the game search, game detail pages, and forum read views accessible without authentication while keeping write operations gated, which is achievable through a restructuring of the Firestore security rules and the Flutter router configuration.

11. References

- Anthropic. (2025). Claude [Large language model]. Available at: https://www.anthropic.com [Accessed 2026].
- Backloggd. (2024). Backloggd Game Tracking Platform. [online] Available at: https://www.backloggd.com [Accessed February 2026].
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001). Manifesto for Agile Software Development. [online] Available at: https://agilemanifesto.org [Accessed March 2026].
- Clegg, D. and Barker, R. (1994). Case Method FastTrack: A RAD Approach. AddisonWesley.
- Dart. (2018). Dart Programming Language. [online] Available at: https://dart.dev/guides [Accessed March 2026].
- Flutter. (2018). Flutter Documentation. [online] Available at: https://docs.flutter.dev [Accessed March 2026].
- Giant Bomb. (2012). Giant Bomb Game Database. [online] Available at: https://www.giantbomb.com/api [Accessed February 2026].
- Google Firebase. (2014). Firebase Documentation. [online] Available at: https://firebase.google.com/docs [Accessed March 2026].
- Google Cloud. (2017). Cloud Firestore Documentation. [online] Available at: https://cloud.google.com/firestore/docs [Accessed March 2026].
- Google Material Design. (2021). Material Design Navigation Guidelines. [online] Available at: https://m3.material.io/components/navigationdrawer [Accessed March 2026].
- HowLongToBeat. (2024). HowLongToBeat Platform. [online] Available at: https://howlongtobeat.com [Accessed February 2026].
- JetBrains. (2020). Kotlin Multiplatform Documentation. [online] Available at: https://kotlinlang.org/docs/multiplatform.html [Accessed February 2026].
- MDN Web Docs. (2024). CrossOrigin Resource Sharing (CORS). [online] Available at: https://developer.mozilla.org/enUS/docs/Web/HTTP/CORS [Accessed March 2026].
- MDN Web Docs. (2024). Responsive Design. [online] Available at: https://developer.mozilla.org/enUS/docs/Learn/CSS/CSS_layout/Responsive_Design [Accessed March 2026].
- Meta. (2015). React Native Documentation. [online] Available at: https://reactnative.dev/docs/gettingstarted [Accessed February 2026].
- MongoDB. (2009). MongoDB Documentation. [online] Available at: https://www.mongodb.com/docs [Accessed February 2026].

- Newzoo. (2023). Global Games Market Report 2023. [online] Available at: https://newzoo.com/resources/trendreports/newzoosglobalgamesmarketreport2023freeversion [Accessed March 2026].
- Node.js Foundation. (2009). Node.js Documentation. [online] Available at: https://nodejs.org/en/docs [Accessed March 2026].
- PlayTracker. (2024). PlayTracker Game Tracking. [online] Available at: https://playtracker.net [Accessed February 2026].
- PostgreSQL Global Development Group. (1996). PostgreSQL Documentation. [online] Available at: https://www.postgresql.org/docs [Accessed February 2026].
- RAWG. (2019). RAWG Video Games Database API. [online] Available at: https://rawg.io/apidocs [Accessed March 2026].
- Statista. (2023). Number of video gamers worldwide 2023. [online] Available at: https://www.statista.com/statistics/748044/numbervideogamersworld [Accessed March 2026].
- Supabase. (2020). Supabase Documentation. [online] Available at: https://supabase.com/docs [Accessed February 2026].
- Twitch. (2015). IGDB API Documentation. [online] Available at: https://apidocs.igdb.com [Accessed February 2026].
- Valve Corporation. (2011). Steam Web API Documentation. [online] Available at: https://developer.valvesoftware.com/wiki/Steam_Web_API [Accessed March 2026].

12. Appendices

Appendix A: GitHub Repository

Full source code and commit history: https://github.com/Kszlufik/FinalProject

Appendix B: Live Application

Deployed application: https://playpalapp123.web.app

Appendix C: Project Landing Page

Project landing page with all submission links: https://kszlufik.github.io/FinalProject/

Appendix D: Firestore Security Rules

The complete Firestore security rules enforce authenticated access on all collections. Users can only write to their own subcollections. Friend requests allow any authenticated user to write to another user's friendRequests subcollection. Forum posts can be created or liked by any authenticated user but deleted only by the post author.

Appendix E: Firebase Cloud Functions

Three Cloud Functions were deployed to Firebase (2nd Generation, uscentral1, Node.js 24): getSteamProfile, getSteamGames, and getSteamAchievements. The Steam API key is stored as the Firebase Secret STEAM_API_KEY and is never accessible from the client.

Appendix F: Composite Firestore Indexes

Two composite indexes were created in the Firebase Console: one on the activity collection indexing uid and timestamp to support the social feed query, and one on the forums posts subcollection indexing likeCount and timestamp to support the top comment ordering.